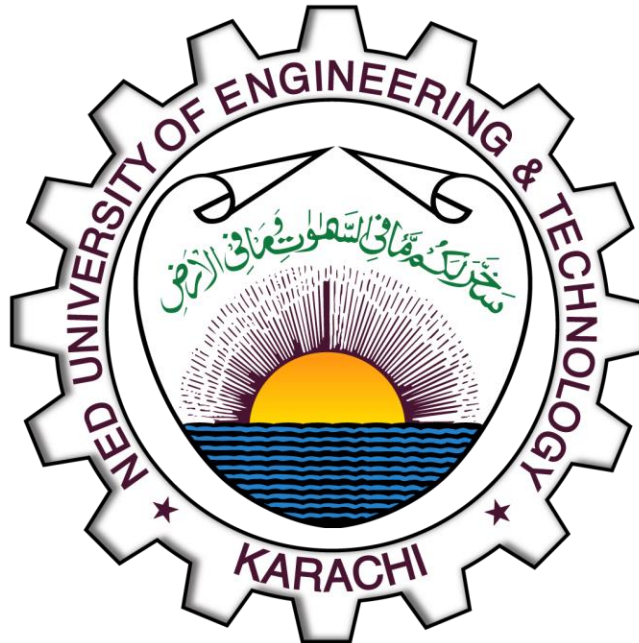


OBJECT ORIENTED PROGRAMMING

[CT-260]



NAME: TAHA AHMED MALLICK

ROLL NO: CT-25183

DEPARTMENT: BCIT **BATCH:** 2025

YEAR & SECTION: FSCS-D

LAB : #11

QUESTION#1:

CODE:

```
#include <iostream>
using namespace std;

class NoDigitsErr
{
public:
    const char *what()
    {
        return "\e[1;31mPassword has no digits!\e[0m";
    }
};

class InsuffLenErr
{
public:
    const char *what()
    {
        return "\e[1;31mInsufficient password length!!\e[0m";
    }
};

class User
{
private:
    string usrName, pass;

public:
    User() : usrName(""), pass("") {}

    bool hasDigits(string pass)
    {
        for (char c : pass)
            if (c >= '0' && c <= '9')
                return true;
        return false;
    }

    void login()
    {
        string input;
```

```

        cout << "Enter your user name: ";
        getline(cin, input);
        usrName = input;
        cout << "Enter your password: ";
        getline(cin, input);
        if (input.length() < 6)
            throw InsuffLenErr();
        if (!hasDigits(input))
            throw NoDigitsErr();
        pass = input;
        cout << "\e[1;32mLogin is successful.\e[0m\n";
    }
};

int main()
{
    User usr;
    try
    {
        usr.login();
    }
    catch (InsuffLenErr &e)
    {
        cout << e.what() << endl;
    }
    catch (NoDigitsErr &e)
    {
        cout << e.what() << endl;
    }
    return 0;
}

```

OUTPUT:

OUTPUT 1:

Enter your user name: Taha Ahmed

Enter your password: abcdefg

Login is successful.

OUTPUT 2:

Enter your user name: Taha Ahmed

Enter your password: abcd

Insufficient password length!!

QUESTION#2:

CODE:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

class Stack
{
private:
    vector<char> stack;

public:
    Stack() {}
    void input()
    {
        string input;
        cout << "Enter any sentence: ";
        getline(cin, input);
        for (char c : input)
            stack.push_back(c);
    }
    void push(char c)
    {
        stack.push_back(c);
    }
    char pop()
    {
        if (!stack.empty())
        {
            char last = stack.back();
            stack.pop_back();
            return last;
        }
        else
            throw "Stack Underflow";
    }
    char peek()
    {
        if (!stack.empty())
            return stack.back();
        else
```

```

        return '\0';
    }
    void view()
    {
        cout << "Stack is filled with: \e[32m";
        for (const char &c : stack)
            cout << c;
        cout << "\e[0m" << endl;
    }
    void reverse()
    {
        string reversed = "", word = "";
        for (const char &c : stack)
        {
            if (c >= 'a' && c <= 'z' || c >= 'A' && c <= 'Z')
            {
                word += c;
            }
            else
            {
                std::reverse(word.begin(), word.end());
                reversed += word + c;
                word = "";
            }
        }
        stack.clear();
        for (const char &c : reversed)
            stack.push_back(c);
        cout << "Your reversed string: \e[1;33m" << reversed << "\e[0m" << endl;
    }
};

int main()
{
    Stack stack;
    stack.input();

    do
    {
        int choice;
        cout << "\nSelect a option:" << endl
            << "1. Push a character" << endl
            << "2. Pop a character" << endl
            << "3. Peek at stack" << endl
            << "4. View Stack Elements" << endl

```

```

        << "5. Reverse each word in stack" << endl
        << "6. EXIT" << endl;
    cout << "What do u want to perform? ";
    cin >> choice;
    cout << endl
        << endl;
    switch (choice)
    {
    case 1:
    {
        char input;
        cout << "Enter a character you want to push: ";
        cin >> input;
        stack.push(input);
        break;
    }
    case 2:
    {
        char popped = stack.pop();
        cout << "Popped element was: " << popped << endl;
        break;
    }
    case 3:
    {
        char peek = stack.peek();
        cout << "Top of stack: " << peek << endl;
        break;
    }
    case 4:
        stack.view();
        break;
    case 5:
        stack.reverse();
        break;
    case 6:
        cout << "EXITING...\n";
        return 0;
        break;

    default:
        cout << "\e[1;31mINVALID CHOICE\e[0m\n";
        break;
    }
} while (true);
return 0;}

```

OUTPUT:

Enter any sentence: How are you? Fine.

Select a option:

1. Push a character
2. Pop a character
3. Peek at stack
4. View Stack Elements
5. Reverse each word in stack
6. EXIT

What do u want to perform? 5

Your reversed string: **woH era uoy? eniF.**

Select a option:

1. Push a character
2. Pop a character
3. Peek at stack
4. View Stack Elements
5. Reverse each word in stack
6. EXIT

What do u want to perform? 3

Top of stack: .

Select a option:

1. Push a character
2. Pop a character
3. Peek at stack
4. View Stack Elements
5. Reverse each word in stack
6. EXIT

What do u want to perform? 4

Stack is filled with: **woH era uoy? eniF.**

Select a option:

1. Push a character
2. Pop a character
3. Peek at stack
4. View Stack Elements

5. Reverse each word in stack
6. EXIT

What do u want to perform? 5

Your reversed string: **How are you? Fine.**

Select a option:

1. Push a character
2. Pop a character
3. Peek at stack
4. View Stack Elements
5. Reverse each word in stack
6. EXIT

What do u want to perform? 1

Enter a character you want to push: !

Select a option:

1. Push a character
2. Pop a character
3. Peek at stack
4. View Stack Elements
5. Reverse each word in stack
6. EXIT

What do u want to perform? 3

Top of stack: !

Select a option:

1. Push a character
2. Pop a character
3. Peek at stack
4. View Stack Elements
5. Reverse each word in stack
6. EXIT

What do u want to perform? 2

Popped element was: !

Select a option:

1. Push a character
2. Pop a character

3. Peek at stack
4. View Stack Elements
5. Reverse each word in stack
6. EXIT

What do u want to perform? 6

EXITING...

QUESTION#3:

CODE:

```
#include <iostream>
#include <algorithm>
#include <chrono>
using namespace std;

class Sorter
{
private:
    int *arr, *myarr;
    const size_t n;

public:
    Sorter(size_t n) : n(n)
    {
        arr = new int[n]();
        myarr = new int[n]();
    }

    void input()
    {
        int input;
        cout << "Enter the elements: \n";
        for (int i = 0; i < n; i++)
        {
            cin >> input;
            arr[i] = input;
            myarr[i] = input;
        }
    }

    void libSort()
    {
        auto start = chrono::high_resolution_clock::now();
        sort(arr, arr + n);
        auto end = chrono::high_resolution_clock::now();
        auto duration = chrono::duration_cast<chrono::microseconds>(end -
start).count();
        cout << "\e[1;32mTime took to sort: " << duration << "micro secs\e[0m\n";
    }
}
```

```

void mySort()
{
    auto start = chrono::high_resolution_clock::now();
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n - i - 1; j++)
            if (myarr[j] > myarr[j + 1])
            {
                int temp = myarr[j];
                myarr[j] = myarr[j + 1];
                myarr[j + 1] = temp;
            }
    auto end = chrono::high_resolution_clock::now();
    auto duration = chrono::duration_cast<chrono::microseconds>(end -
start).count();
    cout << "\e[1;32mTime took to sort: " << duration << "micro secs\e[0m\n";
}

void display()
{
    cout << "Your array: ";
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}

void myDisplay()
{
    cout << "Your array: ";
    for (int i = 0; i < n; i++)
    {
        cout << myarr[i] << " ";
    }
    cout << endl;
}

~Sorter()
{
    delete[] arr;
    delete[] myarr;
}

};

int main()

```

```
{  
    size_t n;  
    cout << "Enter the number of elements: ";  
    cin >> n;  
    Sorter arr(n);  
  
    arr.input();  
  
    cout << "\n\n-----Before Sort-----\n\n";  
    arr.display();  
  
    cout << "\n\n-----After my bubble Sort-----\n\n";  
    arr.mySort();  
    arr.myDisplay();  
  
    cout << "\n\n-----After algorithm Sort-----\n\n";  
    arr.libSort();  
    arr.display();  
  
    return 0;  
}
```

OUTPUT:

```
Enter the number of elements: 7
```

```
Enter the elements:
```

```
9
```

```
0
```

```
-5
```

```
8
```

```
1
```

```
2
```

```
6
```

```
-----Before Sort-----
```

```
Your array: 9  0  -5  8  1  2  6
```

```
-----After my bubble Sort-----
```

```
Time took to sort: 0micro secs
```

```
Your array: -5  0  1  2  6  8  9
```

```
-----After algorithm Sort-----
```

```
Time took to sort: 0micro secs
```

```
Your array: -5  0  1  2  6  8  9
```

QUESTION#4:

CODE:

```
#include <iostream>
#include <map>
using namespace std;

class StudentManage
{
private:
    map<string, string> studentRecord;

public:
    StudentManage() {}

    bool checkGrade(const string &grade)
    {
        string grades[] = {"A+", "A", "B", "C", "D", "F"};
        for (string select : grades)
        {
            if (grade == select)
                return true;
        }
        return false;
    }

    void addStudents()
    {
        string grade, name;
        do
        {
            cout << "Enter name of student or blank to stop: ";
            getline(cin, name);
            if (name == "")
                break;
            do
            {
                cout << "Enter the grade(A+/A/B/C/D/F): ";
                getline(cin, grade);
            } while (!checkGrade(grade));
            studentRecord[name] = grade;
        } while (true);
    }
}
```

```

void viewRecords()
{
    if (studentRecord.empty())
    {
        cout << "-----No Records Found-----\n";
        return;
    }
    cout << "-----Total Records-----\n";
    cout << "S#\tStudents\tGrade\n";
    map<string, string>::iterator itr = studentRecord.begin();
    for (int i = 1; itr != studentRecord.end(); i++, itr++)
        cout << i << "\t" << itr->first << "\t\t" << itr->second << endl;
    cout << "-----End of Records-----\n";
}

void getGrade()
{
    string name;
    cout << "Enter the name of the student: ";
    getline(cin, name);
    if (studentRecord.find(name) != studentRecord.end())
        cout << "\e[1;32mGrade is: " << studentRecord[name] << "\e[0m\n";
    else
        cout << "\e[1;31mThis student doesn't exists.\e[0m\n";
}

void deleteRecord()
{
    string name;
    cout << "Enter the name of the student: ";
    getline(cin, name);
    if (studentRecord.erase(name))
        cout << "\e[1;32mRecord of " << name << " successfully
deleted.\e[0m\n";
    else
        cout << "\e[1;31mThis student wasn't in the records.\e[0m\n";
}

void updateRecords()
{
    string name, grade;
    cout << "Enter the name of the student: ";
    getline(cin, name);
    if (studentRecord.find(name) != studentRecord.end())

```

```

        {
            cout << "Enter the grade(A+/A/B/C/D/F): ";
            getline(cin, grade);
            studentRecord[name] = grade;
        }
        else
            cout << "\e[1;31mThis student doesn't exists.\e[0m\n";
    }
};

int main()
{
    StudentManage students;
    do
    {
        int choice;
        cout << "Select from the options below:" << endl
            << "1. Add students" << endl
            << "2. View records" << endl
            << "3. Get grade of a student" << endl
            << "4. Update record" << endl
            << "5. Delete record" << endl
            << "6. EXIT" << endl;

        cin >> choice;
        cin.ignore();
        switch (choice)
        {
            case 1:
                students.addStudents();
                break;
            case 2:
                students.viewRecords();
                break;
            case 3:
                students.getGrade();
                break;
            case 4:
                students.updateRecords();
                break;
            case 5:
                students.deleteRecord();
                break;
            case 6:
                cout << "EXITING..." << endl;
                return 0;
        }
    } while (choice != 6);
}

```

```

        break;

        default:
            cout << "\e[1;31mINVALID CHOICE\e[0m\n";
            break;
    }
} while (true);
return 0;
}

```

OUTPUT:

```

Select from the options below:
1. Add students
2. View records
3. Get grade of a student
4. Update record
5. Delete record
6. EXIT
1
Enter name of student or blank to stop: Taha
Enter the grade(A+/A/B/C/D/F): A+
Enter name of student or blank to stop: Danial
Enter the grade(A+/A/B/C/D/F): A
Enter name of student or blank to stop:
Select from the options below:
1. Add students
2. View records
3. Get grade of a student
4. Update record
5. Delete record
6. EXIT
2
-----Total Records-----
S#      Students      Grade
1       Danial        A
2       Taha          A+
-----End of Records-----
Select from the options below:
1. Add students
2. View records
3. Get grade of a student
4. Update record
5. Delete record
6. EXIT
3

```

Enter the name of the student: Taha

Grade is: A+

Select from the options below:

1. Add students
2. View records
3. Get grade of a student
4. Update record
5. Delete record
6. EXIT

4

Enter the name of the student: Danial

Enter the grade(A+/A/B/C/D/F): A+

Select from the options below:

1. Add students
2. View records
3. Get grade of a student
4. Update record
5. Delete record
6. EXIT

2

-----Total Records-----

S#	Students	Grade
1	Danial	A+
2	Taha	A+

-----End of Records-----

Select from the options below:

1. Add students
2. View records
3. Get grade of a student
4. Update record
5. Delete record
6. EXIT

5

Enter the name of the student: Taha

Record of Taha successfully deleted.

Select from the options below:

1. Add students
2. View records
3. Get grade of a student
4. Update record
5. Delete record
6. EXIT

2

-----Total Records-----

S#	Students	Grade
----	----------	-------

```
1      Danial      A+
-----End of Records-----
Select from the options below:
1. Add students
2. View records
3. Get grade of a student
4. Update record
5. Delete record
6. EXIT
6
EXITING...
```

QUESTION#5:

CODE:

```
#include <iostream>
#include <set>
using namespace std;

class GuestTracker
{
private:
    set<string> guests;

public:
    GuestTracker() {}

    void addGuests()
    {
        string name;
        do
        {
            cout << "Enter name of student or blank to stop: ";
            getline(cin, name);
            if (name == "")
                break;
            guests.insert(name);
        } while (true);
    }

    void viewGuests()
    {
        if (guests.empty())
        {
            cout << "-----No Guests in the System-----\n";
            return;
        }
        cout << "-----Total Guests-----\n";
        cout << "S#\tName\n";
        set<string>::iterator itr = guests.begin();
        for (int i = 1; itr != guests.end(); i++, itr++)
            cout << i << "\t" << *itr << endl;
        cout << "-----End of Line-----\n";
    }
}
```

```

size_t getCount()
{
    return guests.size();
}

};

int main()
{
    GuestTracker guests;
    do
    {
        int choice;
        cout << "Select from the options below:" << endl
              << "1. Add guests" << endl
              << "2. View guests" << endl
              << "3. Get total count" << endl
              << "4. EXIT" << endl;

        cin >> choice;
        cin.ignore();
        switch (choice)
        {
            case 1:
                guests.addGuests();
                break;
            case 2:
                guests.viewGuests();
                break;
            case 3:
            {
                size_t total = guests.getCount();
                cout << "\e[1;32mTotal Number of guests: " << total << "\e[0m\n";
            }
            break;
            case 4:
                cout << "EXITING..." << endl;
                return 0;
                break;

            default:
                cout << "\e[1;31mINVALID CHOICE\e[0m\n";
                break;
        }
    } while (true);
    return 0;
}

```

OUTPUT:

```
Select from the options below:
1. Add guests
2. View guests
3. Get total count
4. EXIT
2
-----No Guests in the System-----
Select from the options below:
1. Add guests
2. View guests
3. Get total count
4. EXIT
3
Total Number of guests: 0
Select from the options below:
1. Add guests
2. View guests
3. Get total count
4. EXIT
1
Enter name of student or blank to stop: Taha
Enter name of student or blank to stop: Danial
Enter name of student or blank to stop:
Select from the options below:
1. Add guests
2. View guests
3. Get total count
4. EXIT
2
-----Total Guests-----
S#      Name
1       Danial
2       Taha
-----End of Line-----
Select from the options below:
1. Add guests
2. View guests
3. Get total count
4. EXIT
3
Total Number of guests: 2
Select from the options below:
1. Add guests
```

```
2. View guests
3. Get total count
4. EXIT
1
Enter name of student or blank to stop: Taha
Enter name of student or blank to stop: Yousuf
Enter name of student or blank to stop:
Select from the options below:
1. Add guests
2. View guests
3. Get total count
4. EXIT
3
Total Number of guests: 3
Select from the options below:
1. Add guests
2. View guests
3. Get total count
4. EXIT
2
-----Total Guests-----
S#      Name
1       Danial
2       Taha
3       Yousuf
-----End of Line-----
Select from the options below:
1. Add guests
2. View guests
3. Get total count
4. EXIT
4
EXITING...
```